

A PARAMETERIZABLE STANDARD CELL GENERATOR

Examensarbete utfört i Elektroniksystem

vid Linköpings Tekniska Högskola

av

Terese Ekebrand

Nils Funke

Reg nr: LiTH-ISY-EX-3303-2003

Linköping 2003-05-08

A PARAMETERIZABLE STANDARD CELL GENERATOR

Examensarbete utfört i Elektroniksystem
vid Linköpings Tekniska Högskola
av

Terese Ekebrand

Nils Funke

Reg nr: LiTH-ISY-EX-3303-2003

Supervisor: Emil Hjalmarsson

Examiner: Mark Vesterbacka

Linköping, 8 May 2003.



Avdelning, Institution
Division, department
Department of Electrical Engineering
581 83 LINKÖPING

Datum
Date

2003-05-08

Språk
Language

☐ Svenska/Swedish
☒ Engelska/English

☐ _____

Rapporttyp
Report: category

☐ Licentiatavhandling
☒ Examensarbete
☐ C-uppsats
☐ D-uppsats
☐ Övrig rapport

☐ _____

ISBN

ISRN LiTH-ISY-EX-3303-2003

Serietitel och serienummer **ISSN**
Title of series, numbering _____

URL för elektronisk version
<http://www.ep.liu.se/exjobb/isy/2003/3303/>

Titel
Title
En parameteriserbar standardcellgenerator

A Parameterizable Standard Cell Generator

Författare Terese Ekebrand, Nils Funke
Author

Sammanfattning
Abstract

This master thesis describes the creation of a fully parameterizable design tool, intended for automatic generation of standard cell layouts from basic schematic information. The thesis covers general background on programs for automatic layout generation, standard cells and basics in IC design. Algorithms commonly used in various parts of such programs are presented, and the ones used to implement the tool are described in depth.

Nyckelord
Keywords
standard cell, design automation, layout, parameterizable

ACKNOWLEDGMENT

First we would like to thank Mathias Henningsson at the division of Optimization, Linköping University, for his enthusiasm and his valuable help with the algorithms used for global routing. We also would like to thank Emil Hjalmarsson at the division of Electronics Systems, Linköping University, for helping us getting started with skill, and Greger Karlström and Peter Johansson at the same division, for spending time unlocking computers and such for us.

Finally we would like to thank the coffee break crew for many nice coffee breaks. We cherish those moments. A special thank to Deborah Capello for spending her time reading our report, it really helped us making it better.

TABLE OF CONTENTS

1	Introduction	1
1.1	Background	1
1.2	Objectives	1
1.3	Overview of the Report	2
2	Project Basics	3
2.1	Fabrication of Integrated Circuits	3
2.2	CMOS	3
2.3	What Is a Standard Cell?	4
2.4	Rules	5
2.4.1	Design Rules	5
2.4.2	Standard Cell Rules	7
2.5	Physical Design Automation Programs	8
2.5.1	Partitioning	8
2.5.2	Floor Planning and/or Placement	8
2.5.3	Pin Assignment	9
2.5.4	Routing	9
2.5.5	Compaction	9
3	Standard Cell Model Formulation	11
3.1	Placement Model	11
3.2	Routing Model	12
3.2.1	Grid-Based Versus Gridless Routing Model ..	12
3.2.2	Reserved Versus Unreserved Layer Model ..	13
3.2.3	Applied Routing Model	14
4	Conceptual Design of the Cell	17
4.1	General	17
4.2	Partitioning	18
4.2.1	Common Partitioning Algorithms	18
4.2.2	Applied Partitioning Algorithm	19
4.3	Placement	21
4.3.1	Common Placement Algorithms	21
4.3.2	Applied Placement Algorithm	25
4.4	Global Routing	29
4.4.1	Some Approaches on Global Routing	29

4.4.2	Used Approach on Global Routing	30
4.5	Pin Placement.	35
5	Detailed Level Design of the Cell	37
5.1	General.	37
5.2	Placement.	38
5.3	Routing.	39
5.3.1	River Routing.	39
5.3.2	Detailed Routing in This Tool.	41
6	Results	45
6.1	The Resulting Program	45
6.2	Test Results	48
7	Conclusions	
7.1	Summary	53
7.2	Discussion of the Result and Possible Improvem.	54
	References	57

1

INTRODUCTION

1.1 BACKGROUND

This project has been carried out at the division of Electronics Systems, Department of Electrical Engineering (ISY), Linköping University.

A standard cell is a small layout that function as a building block in larger layouts (IC:s). Standard cells are widely used when creating large layouts. Since the technologies change quite often there is always a need for new standard cells. Normally the creation of a new standard cell library requires a great design effort in terms of man-hours because a cell library may contain 150 or more different instances. Therefore, a design automation program for standard cells could drastically lower the design effort, especially if such a program were written so that it could easily be reused in new technologies.

1.2 OBJECTIVES

The aim of this project is to create a program in the Cadence layout language, SKILL, that is able to generate a layout of a standard cell based on basic schematic information, i.e., the information on the transistors and their interconnections. The program should not be restricted to only use one particular technology, on the contrary it should be as flexible as possible. The user should be able to use the program to create cells with different driving capability and to specify the width and length of individual transistors. For a brief introduction to the SKILL language, we refer the reader to [2].

1.3 OVERVIEW OF THE REPORT

Chapter 2 introduces different concepts used in the thesis. In Chapter 3, the abstract model used for the standard cells is formulated. The design and implementation of the tool is carried out in two main steps. First a high level plan of the cell is derived and then a detailed layout is done. In Chapter 4, algorithms for the high level design are presented and which algorithms that are used are specified. In Chapter 5 algorithms that can be used for the detailed level design are presented, and the algorithms that are used are specified. The results of the project are presented in Chapter 6. Finally a summary and a discussion of the results and possible improvements are given in Chapter 7.

2

PROJECT BASICS

2.1 FABRICATION OF INTEGRATED CIRCUITS

An integrated circuit, often referred to as an IC, is a semiconductor wafer on which thousands or millions of tiny resistors, capacitors and transistors are fabricated. An IC can function as an amplifier, oscillator, timer, counter, computer memory, or microprocessor, etc.

An IC consists of several layers of different materials. The shape, size, and location of material in each layer must be accurately specified for proper fabrication. A specification of all geometric shapes that are needed in a certain layer must exist. The specification is then used to create patterns of each material in a sequential manner, resulting in a complex pattern of several layers. Devices are specified by overlapping a shape in one layer with another shape in a different layer.

2.2 CMOS

CMOS is short for Complementary MOS (where MOS is short for Metal Oxide Semiconductor), meaning a circuit that is realized with a pMOS network connected to positive supply, and an nMOS network connected to negative supply. Both networks are connected at the output. In Figure 2.1, a CMOS NOR2 gate is shown. If the pMOS network is conducting, i.e., both V_a and V_b are low, the nMOS network is not. The opposite is also true, if the nMOS network conducts the pMOS does not, resulting in that for any given

input combination the output, V_{out} , is either connected to positive or negative supply via a low-resistance path. Also a DC current path is never established between positive and negative supply since the two networks never conduct at the same time [4]. Any circuit that reminds of this design style is in this thesis referred to as a CMOS structure.

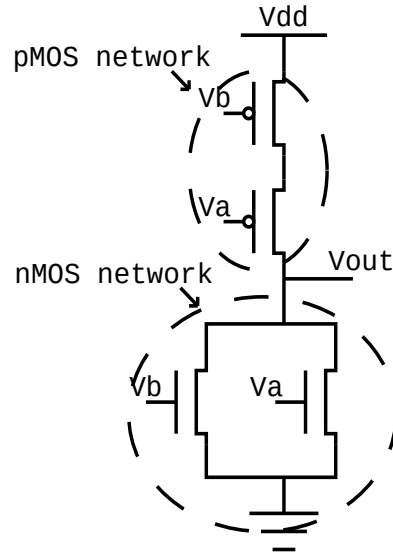


Figure 2.1: A CMOS NOR2-gate

2.3 WHAT IS A STANDARD CELL?

A standard cell is a small circuit layout designed with a number of constraints. It is used as a building block in larger layouts and is therefore designed to be practical to use in big arrays which means that cells with different functions and layouts have to fit together. Standard cells typically use only one metal layer for connections inside the cells, so that the other layers can be used for over-the-cell routing. A typical standard cell can be seen in Figure 2.2. In the figure some different features of the cell are marked and explained. These are the supply wires at the top and bottom of the cell. The positive power supply line is located at the top. P-transistors are placed at the top and the n-transistors at the bottom. Also the abutment box that encloses the cell is marked. This is only a fictitious border around the cell that defines in what area to place different objects.

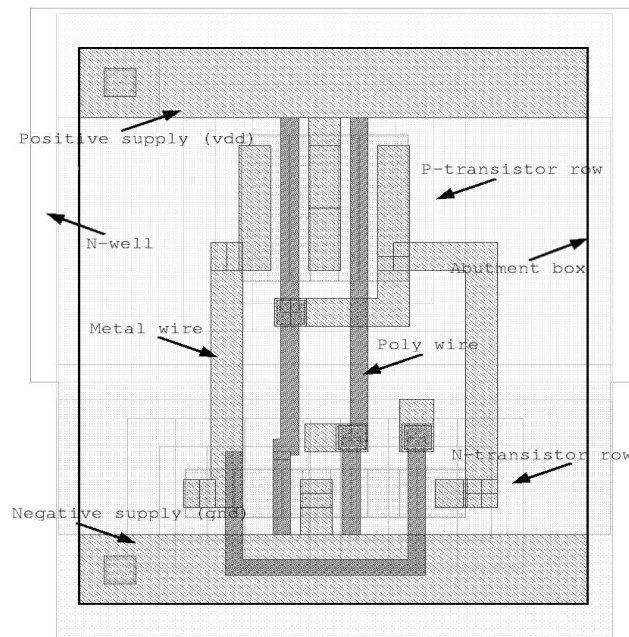


Figure 2.2: A standard cell layout showing some typical features

2.4 RULES

When designing a standard cell a large set of rules must be followed. These can basically be divided into two main categories, *design rules* and *standard cell rules*.

2.4.1 DESIGN RULES

The constraints imposed on the geometry of an integrated circuit layout in order to guarantee that the circuit can be fabricated with an acceptable yield are called *design rules*. These rules are unique for each technology and can differ considerably between different technologies. The purpose of design rules is to prevent faulty layouts. More specifically, design rules are introduced to preserve the integrity of topological features on the chip and to prevent isolated features from short-circuiting each other. Usually when a new technology is being created, the design rules need to be re-established [6].

There are three types of design rules:

- **Size Rules.** Size rules specify the minimum feature sizes on different layers to ensure a valid design of a circuit. Examples of different size rules are minimum widths and minimum areas for objects in different layers. Interconnect lines usually run over rough surfaces, unlike the smooth surface over which active devices, such as transistors, are patterned. Therefore the minimum feature size used for interconnects is somewhat larger than that used for active devices.
- **Separation Rules.** Different objects on the same layer or in different layers must be separated from each other. The separation distances are the same as the minimum widths for interconnect wires, primarily because that makes it easy to maintain a good interconnect density. Most technologies have a spacing rule for all layers.
- **Overlap Rules.** Overlap rules exist in order to protect the layout against mismigration of poly and diffusion, which can result in a short-circuited channel in a transistor, or mismigration of vias between different layers, which can result in two objects being connected that should not be, or two objects not being connected that should be.

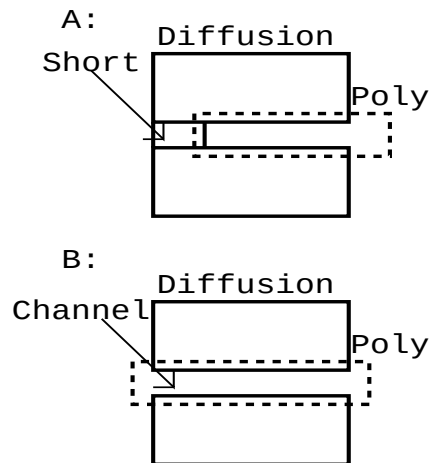


Figure 2.3: A: Effect on a transistor channel by a mismigrated gate-poly
B: Correctly formed channel

There exist rules other than the ones mentioned above, but these are not relevant for this thesis, since they cover subjects such as how to place bonding pads, etc., which is not done in this project.

It is also common that a layout is designed with widths and separations that are larger than the minimum rules presented above. This could for example be of performance reasons, e.g., a long wire should maybe be designed with twice the minimum width to keep its resistance at an acceptable level.

Another design specific parameter is the *manufacturing grid*. When an IC is fabricated, the fabrication process uses a grid with a certain resolution. All objects in the design must be placed on this grid, otherwise the IC can not be properly manufactured.

2.4.2 STANDARD CELL RULES

When designing a standard cell, certain rules must be followed to ensure the conformity of the cells made [8]. The rules are of the following types:

- **Naming Conventions.** Supply wires must be named the same in all the cells. In the cells developed in STMicroelectronics 0.18 μm these names are 'vdd!' and 'gnd!'. Pin labels must follow certain rules, e.g., use a certain layer, have a certain font of a certain size etc.
- **Cover Layers.** The cells created must have certain areas covered by certain layers. All cells must have, for example, the upper part of the area covered by an n-well layer.
- **Enclosure of Abutment Box.** Most layers in the cell must be a certain distance, commonly half the separation distance, from the abutment box in order to ensure that the cells can be placed next to each other in all directions without any design rules being violated. For example all the objects in the poly layer must lie at least one half minimum poly separation distance inside the abutment box, if different cells shall be able to be placed next to each other. Other layers must enclose the abutment box, e.g., the n-well, this so that it is certain that the n-well of an entire row of standard cells becomes one single feature.
- **Cell Height.** All cells have a fixed height so that they can be placed next to each other in rows. Sometimes, multiples of this height are allowed.

2.5 PHYSICAL DESIGN AUTOMATION PROGRAMS

As mentioned before the aim of this project is to create a design automation program for creation of standard cells. A design automation program is a program intended to automate the process of converting a specification of an electrical circuit into a layout. A specification could be a schematic view of an integrated circuit, or a program written in a hardware description language such as VHDL. The task is to place and route a number of blocks. In this project the blocks are transistors, but generally the blocks can be of any type, e.g., standard cells. The process can be divided into a series of steps [6].

2.5.1 PARTITIONING

Partitioning means to divide the design into several smaller components or blocks. An example of this might be the partitioning of an AND-gate into a NAND-gate and an inverter. For small circuits, this might not be necessary.

2.5.2 FLOOR PLANNING AND/OR PLACEMENT

The task in this step is to decide where to place the different components or blocks. If the blocks have a flexible shape, this phase is known as floor planning. The placement problem can be viewed as a restricted version of the floor planning problem. A placement problem might be to order a set of transistors with a fixed rotation. A floor planning problem occurs when we allow rotation of the transistors. To simplify placement and floor planning, the blocks that are placed are often assumed to be rectangular. In floor planning, the aspect ratio between the width and the height of the blocks is laborated with. Other shapes have been dealt with, e.g., L- and T-shapes, but such blocks tend to make the algorithms computationally intensive. A placement and floor planning algorithm often takes the following three factors into consideration.

- *Layout area.* As small layout area as possible is often one of the criterions to fulfill.
- *Completion of routing.* A bad placement can result in a layout that is hard or impossible to route.
- *Circuit performance.* By choosing a placement that minimizes the longest path in the net, a shorter total delay of the circuit can be achieved.

2.5.3 PIN ASSIGNMENT

When the blocks have been placed, a pin assignment is usually done for each block. This means that one of several equivalent pin candidates is chosen as pin for each signal going in to or out from a block. This is done so that the routing becomes as easy as possible.

2.5.4 ROUTING

The routing can often be broken up into two steps, global and detailed routing. In the placement phase, the exact locations of circuit blocks and pins are usually determined. Space not occupied by the blocks can be viewed as a collection of regions, called routing regions. In global routing each net, a set of nodes that shall be connected, is assigned a number of routing regions without specifying the actual geometric layout of the wires themselves. Some nets might have timing constraints, and these can be compared with the estimated delay of the nets based on their lengths from the global routing phase. If the constraints are not met, the net might have to be ripped up or the global routing phase repeated. After global routing, each net has been assigned a pin pair in all the regions it traverses. It is then up to the detailed router to do the actual layout of each net.

2.5.5 COMPACTION

In the compaction step the layout is parsed and all objects are moved as close to each other as possible. In this project no compaction is done, since the cell height is decided from the beginning and the transistors are horizontally placed as close to each other as possible already from the beginning. For more information on compaction, see [6].

3

STANDARD CELL MODEL FORMULATION

The problem of automatically generating a standard cell layout is very complex. It is therefore of great importance to simplify and strictly formulate a model for the problem, and already from the beginning remove some degrees of freedom from the design procedure.

In this chapter, the model and what kind of restrictions and simplifications it is based on are presented.

3.1 PLACEMENT MODEL

First a basic floor plan is decided upon, in order to make the problem of placement and routing more manageable. The transistors in the cell are placed in two parallel rows, the p-transistors above the n-transistors, with their gates directed vertically.

The decided floor plan leads to that the placement only has to deal with ordering and source drain rotation of the transistors in each row. Transistors are therefore, before they are actually placed in the layout, simply modelled as three evenly spaced nodes. The cell model can be studied in Figure 3.1.

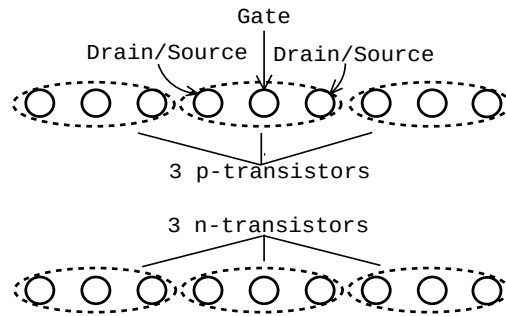


Figure 3.1: Model of the transistor rows

When the layout is done, the different sizes of the transistors need to be taken into consideration so that a proper layout can be generated. Apart from this, the model needs some rules to do the layout by, besides the design rules. These are decided to be:

- Substrate contacts shall always be placed at the lower and upper left corner respectively.
- All transistors in the p-transistor row shall have their upper parts horizontally aligned. The opposite holds for the n-transistor row.
- M-Poly contacts, i.e., contacts connecting the first metal and the poly layer, and diffusion contacts shall be placed as close to the transistor as possible.
- Notches can be ignored (notches can easily be attended to by the user).
- Pins can be placed as arbitrary shapes, but only in the metal layer used in routing.

3.2 ROUTING MODEL

3.2.1 GRID-BASED VERSUS GRIDLESS ROUTING MODEL

When choosing a routing model, the first thing to consider is whether to use a grid-based or a gridless model. A grid-based model is a model where all wires are placed on a certain grid. The grid lines must be enough separated for wires to be able to run in parallel with each other without violating any separation rules. This model type results in easily computed problems but has

many disadvantages. One of these is that the terminals always have to be on grid, and since the terminals in this project are the drains, gates and sources of the transistors that would demand the transistors to be evenly spaced. A gridless model on the other hand does not use a grid and hence, transistors can be arbitrarily spaced. This model is however computationally harder than the grid-based one. The two models can be seen in Figure 3.2.

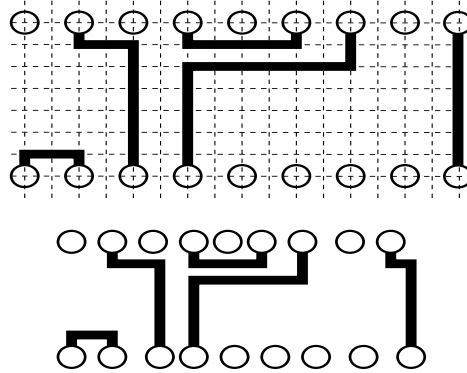


Figure 3.2: Grid-based vs. gridless routing model

3.2.2 RESERVED VERSUS UNRESERVED LAYER MODEL

The next thing to consider is whether to use a reserved layer model or an unreserved layer model. A reserved layer model is a model where either the vertical, the horizontal or both types of segments of a wire are restricted to use some particular layers. In the implementation presented in this thesis only two routing layers are allowed and this would mean that all vertical segments would be restricted to one layer and all horizontal segments to the other. An unreserved model is a model where any segment of a wire can be placed in any layer, used for routing. Most of the existing routers use reserved layer models, probably since they give less complex routing problems and are faster. However a reserved layer model leads to solutions containing large numbers of vias. There are several reasons to minimize the number of vias in a layout.

- A chip with more vias has a smaller probability of being fabricated correctly.
- Every via has an associated resistance which affects the circuit performance.

- The size of the via is sometimes larger than the width of the wires resulting in that more vias leads to more space needed for routing [6].

The unreserved layer model also has the advantage that wires of two different layers can lay on top of each other and therefore need no more space than one of the wires in the reserved layer model. A comparison between the space needed for the two different models can be seen in Figure 3.3.

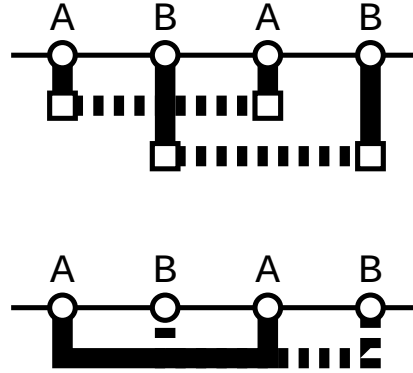


Figure 3.3: Reserved vs. unreserved layer model

3.2.3 APPLIED ROUTING MODEL

Since one of the goals of this project is to make the cell as compact as possible, the grid-based model is not a good approach. Hence a gridless model is used, which means that wires and terminals, and thereby also devices, can be placed arbitrarily.

To keep the number of vias low and to minimize the channel height, the layer model used is a slightly modified version of the unreserved layer model, even though the problems associated with this model are computationally harder than the ones for the reserved layer model. The modification means that the model only allows usage of one layer for each wire.

The routing is carried out above the p-transistor row, below the n-transistor row and in the channel between the two rows. However, there are some restrictions on which types of wires that are allowed to be drawn. It is decided that all contacts between the two routing layers, the M-Poly contacts, shall be restricted to the channel and be placed close to the transistors in order to remove some degrees of freedom from the routing problem and also to sim-

plify the layout later on. This means that any wire that is drawn in a layer that differs from any of the layers of the two terminals it connects is restricted to the channel. It also results in that no wires can shift layers in the middle. This has a negative effect on the routability, but on the other hand the global routing problem is easier to formulate.

Supply wires can be placed in one of two metal layers, where one of the layers is the one used for routing.

Finally it should be noted that gate nodes must not be defined as supply nodes by the user, since that would require a contact outside the channel and hence violate previously stated rules, regarding contact placement.

4

CONCEPTUAL DESIGN OF THE CELL

In this chapter, algorithms for high level design of the cell are discussed.

4.1 GENERAL

In this chapter, the high level model discussed in Chapter 3 is used. Hence, each of the transistors is modelled as an object with only three coherent nodes, and these objects are placed in two rows, one in the upper part of the cell, all the p-type transistors, and one in the lower part, all the n-type transistors. Based on this description, partitioning, placement, pin placement, and global routing are done.

All objects treated in this chapter, such as wires and transistors, are represented, in the tool, by objects with a number of properties. The properties for a transistor are such things as rotation (drain-source or source-drain), transistor type, node names, etc. Properties for a wire are what layer it is drawn in, between which nodes it runs, etc. At the end of this phase, the transistor and wire objects do also contain information on where contacts are needed and which objects that act as pins.

The following text describes algorithms that can be used for placement, partitioning, and routing, and also which of these that were selected for the implementation.

4.2 PARTITIONING

Efficient design of a complex system calls for decomposition into smaller subsystems. Each of these subsystems can be designed independently and simultaneously to speed up the design process. This decomposition step is called partitioning.

In this thesis, rather small designs are considered, which might lead to the conclusion that no partitioning is necessary. However, even a small design is easier to process if it is partitioned.

4.2.1 COMMON PARTITIONING ALGORITHMS

There are some issues that must be regarded when partitioning a system. Firstly, the original functionality of the system must be preserved after partitioning. Secondly, the interconnections between any two subsystems should be minimized. Finally, the partitioning should not take more than a fraction of the total time spent on the design process [6].

The problem of minimizing the interconnections between partitions is called the mincut problem. This problem is NP-complete. Therefore a number of heuristic algorithms for solving the problem exists.

There are two important categories of such algorithms:

- Group migration algorithms.
- Simulated annealing and evolution based algorithms.

The group migration algorithms start with a fixed number of partitions, usually randomly generated, and then try to improve the partitioning by moving components between the partitions. This class of algorithms are usually quite efficient, but the number of partitions must be known from the outset. Group migration algorithms are deterministic [6].

The simulated annealing or evolution algorithms use a cost function and a set of moves to find a solution. Unlike the group migration algorithms, these algorithms accept moves that deteriorate the current solution. The algorithms start with a random solution and, as they progress, the proportion of adverse moves decreases. Deteriorating moves are allowed in order to avoid being trapped in local minima [6] and [7].

4.2.2 APPLIED PARTITIONING ALGORITHM

In the tool none of the above methods are deployed for partitioning. Instead partitioning is based on the assumption that good partitions are equivalent with the CMOS structures (described in Chapter 2.2), that can be found in the cell. The full adder in Figure 4.1 will be divided into four different partitions, as can be seen in the figure.

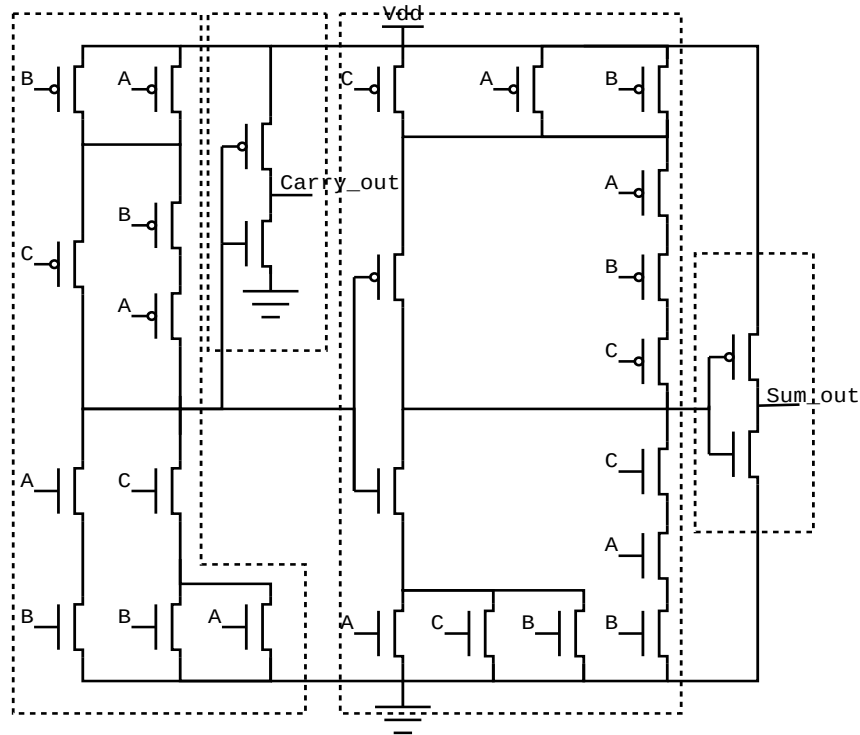


Figure 4.1: A full adder with its partitions shown

The schematic presented to the program by the user can be in total disarray. This means that a way to find the CMOS structures must be developed. The algorithm used is based on the observation that two different CMOS structures only are connected via source-source at supply nodes or via drain-gate connections. If all such connections are removed from a schematic the CMOS structures become isolated from each other. See Figure 4.2.

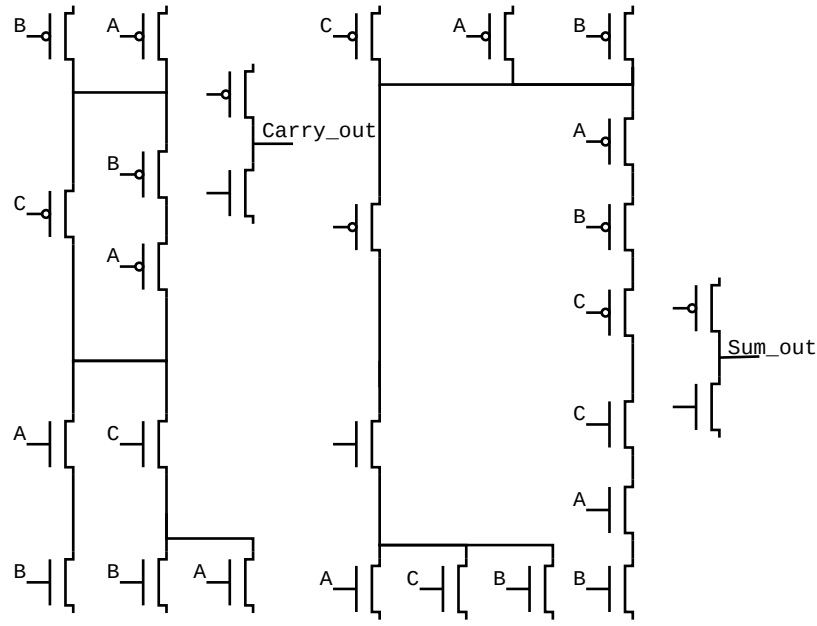


Figure 4.2: A full adder with its CMOS structures separated

The algorithm conducts a depth-first search. It starts with a list containing all transistor objects in the circuit. The transistor objects found during the search are put in a partition and are then removed from this list. Every search for a new partition will start with a transistor object connected to positive supply, hence this will be the root of a tree. To find the next transistor object to put in the tree, the node on the opposite side of the transistor is identified, the one that is not positive supply, and the first transistor object in the list that is connected to this node is taken. Each transistor object found will be put in the tree as a new vertex and is removed from the transistor list. The branches are cut when a supply node is reached. The result for the CMOS structure in Figure 4.3 A will be a tree as shown in Figure 4.3 B, where the vertices have been found in the order indicated by the numbers to the upper left of each vertex.

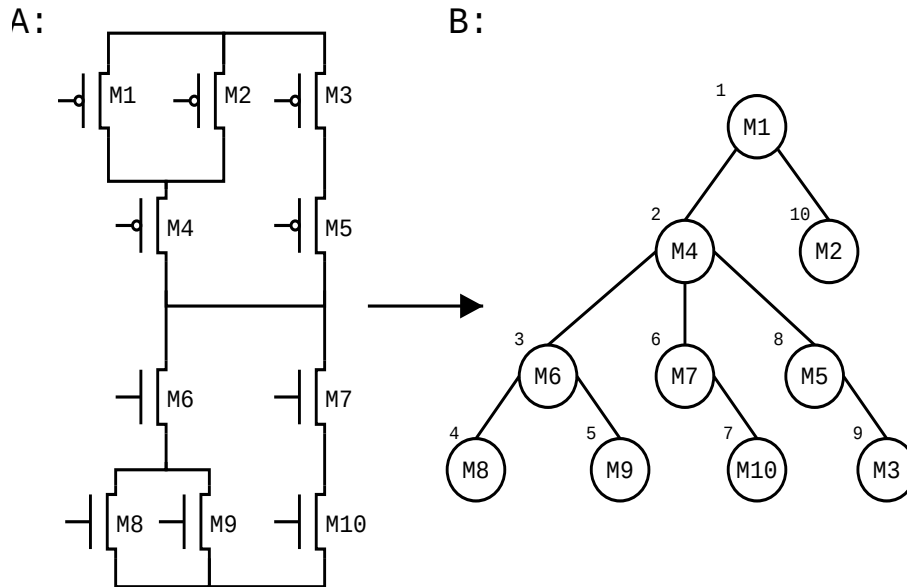


Figure 4.3: A CMOS circuit with its resulting tree structure

It should be added that if there remain transistors not assigned to any partition, which might be the case if none of the transistors in the cell are connected to the supply (not a CMOS structure), the same procedure will be performed, but starting in an arbitrary node represented among the remaining transistors, until all transistors have been assigned to a partition.

4.3 PLACEMENT

A good placement is the key to a good layout. Poor placement leads to larger area consumption, performance degradation and a layout that is harder to route.

4.3.1 COMMON PLACEMENT ALGORITHMS

There is a number of approaches on placement. The most successful ones belong to the simulation based class of placement algorithms, i.e., the class of algorithms based on simulation of natural phenomena. Other approaches yielding successful results are integer programming based algorithms. There are also other algorithms not belonging to any of these two groups, e.g., the

partitioning based algorithms. Only simulation based algorithms will be discussed here, since they are commonly used and yield good results. For information on the other algorithm classes, see [6].

The simulation based algorithms for placement can be divided into three sub-categories: simulated evolution, force directed placement, and simulated annealing.

SIMULATED EVOLUTION

Simulated evolution tries to simulate the evolutionary process in nature. The algorithm will start with an initial set of placement configurations, called the population. The algorithm is iterative. In each iteration, called a generation, a fixed number of individuals exists. During each iteration these individuals are evaluated on the basis of certain fitness tests, that can determine the quality on each placement. Offspring is then produced by selecting two parents from the population, with a probability proportional to their fitness, and let their genes combine into the offspring. This combination is done by crossover, i.e., chosen parts from the parents schemata are combined in the offspring. Afterwards, the resulting schemata of the offspring might be mutated, i.e., random changes to the resulting schemata of the offspring might be generated. When a number of offsprings is generated, a new generation is picked from both the parents and the offspring. The selection is done so that the probability to survive, i.e., to pass on to the next generation, is proportional to the fitness of each individual, and so that the number of individuals in each generation is kept constant.

FORCE DIRECTED PLACEMENT

Force directed placement uses the similarity between placement problems and classical mechanics. The components are modelled as a group of bodies connected to each other by springs. Springs are placed between the components that are connected to each other so that these bodies exert an attractive force on each other. Blocks not connected to each other should exert a repelling force on each other so that they do not end up overlapping each other. The bodies will now move until equilibrium is reached. Some of the bodies must be anchored for the algorithm to work. The problem formulated in this way can be solved using methods from classical mechanics.

SIMULATED ANNEALING

In condensed matter physics, annealing denotes a physical process in which a solid in a ‘heat bath’ is heated up to a maximum value at which all particles of the solid arrange themselves randomly in the liquid phase, followed by cooling by slowly lowering the temperature of the heat bath. If the maximum temperature is high enough and the cooling is done slow enough the particles in the solid will arrange themselves in the low energy ground state of a corresponding lattice. The annealing process can be described as follows. At the maximum temperature and at each lower temperature, T , during the cooling process, the solid is allowed to reach thermal equilibrium. In thermal equilibrium, the probability for a system of being in a state with energy, E , is proportional to the Boltzman factor:

$$e^{-\frac{E}{K_B \cdot T}}$$

where E is the energy, K_B is the Boltzman constant and T is the temperature. As the temperature is lowered, the probability for a system to be in a state with high energy decreases, and when the temperature approaches zero only the lowest energy states have a probability differing from zero. However, if the cooling is done too rapidly, i.e., if the system is not allowed to reach thermal equilibrium for each temperature value, defects can be ‘frozen’ into the structure.

To simulate the process of reaching thermal equilibrium at a given temperature, T , a *Monte Carlo method* is deployed. This works in the following way. Given the current state of the system, characterized by the position of its particles, a small, randomly generated perturbation is applied by a small displacement of a randomly chosen particle. If the ratio between the Boltzman factors of the first and second state is greater than one the change has resulted in a lower energy for the overall system and it is therefore accepted with a probability of one. Otherwise the change is accepted with the probability of the ratio. This makes sense since the ratio of the Boltzman factors reflects the probability that the system changes from a state to another.

The simulated annealing algorithm is a series of the algorithm described above executed as a sequence of decreasing temperature. Given a neighbourhood structure, i.e., all the states available from the current state, simulated annealing continuously attempts to transform the current configuration into one of its neighbours. This is best described as a Markov chain, a sequence of

trials where the outcome of each trial depends only on the outcome of the previous one, i.e., the current configuration. From this, two different formulations of the simulated annealing algorithm can be distinguished, one where each (infinite) Markov chain is generated at a fixed temperature and the temperature is decreased between subsequent Markov chains, and one where the temperature is decreased between each subsequent transition. It is of course impossible to achieve infinite Markov chains in practice, so usually one resorts to an implementation with finite Markov chains generated at stepwise decreased temperature values. This results in four parameters that should be specified.

- An initial value of the control parameter (temperature).
- A final value of the control parameter (temperature).
- A length of the Markov chains.
- A decrement rule for the control parameter (temperature).

Any certain choice of these parameters is called a cooling schedule. The choice of the initial value for the control parameter is often based on the argument that equilibrium can be achieved by choosing the value for the control parameter so that virtually all transitions are accepted. Because of this all configurations have the same probability so that every configuration of the system is in equilibrium. A good value for the parameter can be found by assigning a large value to the parameter and perform a couple of transitions. If the acceptance ratio is less than a given value (normally about 80%), double the current value of the control parameter. Other similar methods for finding an initial value exist. A stop criterion might be that the algorithm has been executed for a fixed number of values of the control parameter, or that the final configurations after a number of Markov chains are the same. The last criterion can be combined with a criterion for the acceptance rate. The length of the Markov chains can be adjusted to the size of the problem so that all chains are of equal length for all values of the control parameter. Another approach is to have the length dependent on the number of accepted configuration changes i.e., a minimum amount of transitions is to be accepted in every Markov chain, but as the control parameter approaches zero the acceptance ratio also approaches zero, which means infinite Markov chains and because of this approach must be combined with a ceiling for the length of the Markov chains. A number of other approaches exists for this parameter choice. The most common way to decrement the control parameter is to let the next value of the parameter be the product of a constant smaller than but

close to one and the current value. This means that a constant ratio between two adjacent values is kept. The difference between adjacent values could also be kept constant.

An easy way to explain simulated annealing is to study a cost function of one variable. This function probably has a global minimum as well as several local ones. In order not to fall down into a local minimum and stick there (as would be the case if an iterative improvement algorithm was deployed), it is needed to be able to climb uphill. In artificial intelligence applications the simulated annealing algorithm is known as hill climbing. This is achieved by accepting cost increases with some probability higher than zero. As this probability decreases, the chosen solution closes in on the global minimum. The fact that we have convergence can be proved. It has also been shown that for at least some problems simulated annealing performs better than iterative improvement (for some different starting configurations). The origin of simulated annealing is the Metropolis algorithm. For a more detailed description refer to [6], [7].

4.3.2 APPLIED PLACEMENT ALGORITHM

The placement done in this thesis is based on the simulated annealing algorithm. This was decided because simulated annealing along with simulated evolution is one of the most well documented algorithms in this field and it has yielded good results. It was selected over simulated evolution partly because it is easier to implement and also because simulated evolution uses larger amounts of memory since it keeps track of the individuals between generations.

In motivation of the algorithm presented later, the following calculation is done. Assume that the full adder in Figure 4.1 shall be created. This particular circuit has 28 transistors, 14 of p-type and 14 of n-type. These transistors will be partitioned into 4 partitions, two inverter partitions and one containing the first ten transistors and one the remaining fourteen. If these partitions were to be placed in a row the following situation arises. In the row the number of ways to place the partitions is $4!$. Each of the transistors can be rotated, meaning that each of them can be chosen in two ways introducing another factor of 2^{28} for the entire row. The order in which the transistors can be placed in the partitions is for the inverters $1! * 1!$. For the other two partitions it is $5! * 5!$ and $7! * 7!$ respectively, since each partition is made up of equally many n-transistors and p-transistors whose order can be chosen arbitrary. Hence the

possible number of configurations is $2^{28} * 4! * 1! * 1! * 1! * 1! * 5! * 5! * 7! * 7! = 2.3565 * 10^{21}$. Obviously an evaluation of all these configurations can not be done in reasonable time, therefore the simulated annealing algorithm is deployed, in order to find an acceptable solution in reasonable time. During tests with simulated annealing it became apparent that also this algorithm took quite some time to find a usable solution when all transistors were moved and rotated. So instead of considering every transistor, each partition is assigned a number of different placements which thereafter are used as blocks in simulated annealing.

The internal placement of each partition is done in such a way that the number of gaps between the transistors is minimized. This has two effects. Firstly, very bad transistor configurations where almost all transistors need wires drawn to their sources and drains are eliminated from the possible solutions. Secondly, the layout will be more compact as the transistors can be placed closer to each other. In case of the full adder this means that we have four partitions where the two inverters have $2 * 2$ different placements each in the cell. The other two partitions will be sorted in $84 * 84$ and $20 * 20$ ways respectively. The result of this is that the possible number of configurations is drastically reduced to $4! * 20^2 * 84^2 * 2^2 * 2^2 = 1.0838 * 10^9$.

The algorithm used when finding good placements for the partitions is not earlier described in the literature studied for this thesis. The algorithm divides each partition into one p-part and one n-part, and then the two parts are placed separately. In order to make the layout as compact as possible, the transistors should be arranged in a way that the first node of each transistor (that is not a start transistor) is the same as the last node of the preceding transistor. To attain this, a graph search is done where the graph represents the circuit and the transistors become edges and the interconnections, i.e., the nodes, become vertices. Such a graph is shown in Figure 4.4.

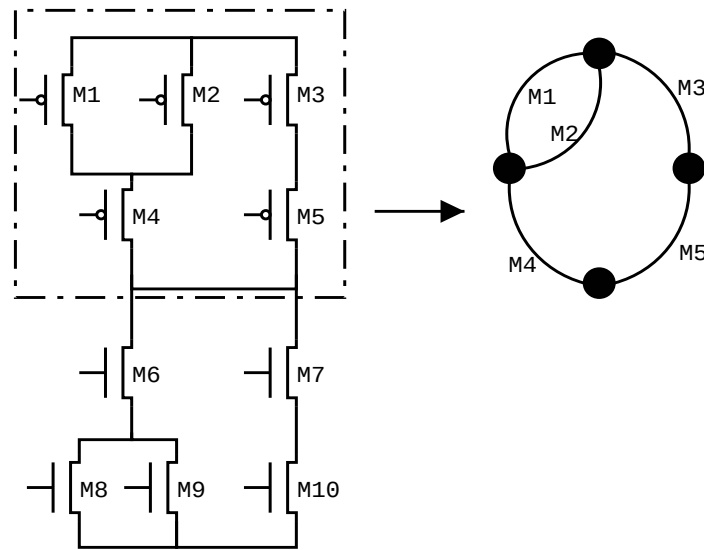


Figure 4.4: A circuit and the corresponding graph of the p-part

The problem of placing the transistors is now equivalent with finding a path through the graph that traverses all the edges once and only once. This is known as the problem of the bridges in Königsberg, and was addressed by Euler [1]. A path that traverses a graph using all the edges once and only once, is called an Euler circuit when it starts and ends in the same node and an Euler path otherwise. In Figure 4.5 an Euler path in the graph from Figure 4.4 can be seen.

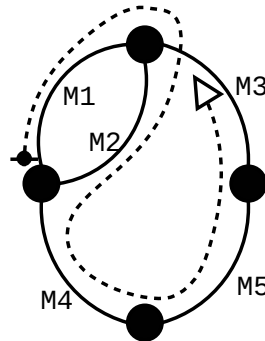


Figure 4.5: The graph with one possible Euler path drawn

It should be noted that at most two nodes in a graph are allowed to be of une-

ven degree, i.e., have an uneven number of edges connected to them if an Euler path is to exist [1]. In Figure 4.6 a graph without Euler paths is shown.

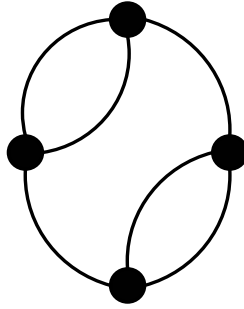


Figure 4.6: A graph lacking Euler paths

In the tool, all possible Euler paths of a partition are of interest. These will then be used as different placement suggestions for that particular part in the simulated annealing procedure. In order to find all possible Euler paths (and put them in lists) the following is done. First, a vertex in the graph is chosen and one list for each edge connected to this vertex is built. For each of these lists, one new list for each edge connected to the vertex in the opposite end of the edge in the current list is built, unless the edge is not already in the current list. The lists do now contain two edges. For each of these lists a new list is built for each edge connected to the vertex that is in the opposite end of the edge last put in the list. This continues until all the lists are either full (contain all edges of the graph) or are in a dead end (all the edges connected to the last vertex are already in the list). The lists that end in a dead end are of course not saved. This procedure is then done for all vertices in the graph, which guarantee that all possible Euler paths are found. At the end all the edges (transistors) are rotated in such a way that they, seen as a row, fit together. Parts of the algorithm can be found in [4].

If no Euler paths exists, another type of search is done. This search procedure aims at finding different structures in the graph, such as parallel structures or serial structures. These structures are searched for in a certain order. First all parallel structures are sought, and the transistors in each such structure are put together, forming a new object that can be treated as one single transistor. After this, all transistors and objects located in serial structures are found and are thereafter also treated as objects, one for each serial structure. This is followed by a search for other structures. The search sequence will loop until all transistors are collected into just one object.

This procedure will, just as the first algorithm, in most cases render more than one solution, since there often is more than one possible way of ordering the transistors and objects in each object. However, all possible high quality solutions (highly compact solutions) will not be found, since the algorithm always starts with a search for parallel structures. If it instead for instance started with looking for serial structures, other, equally good, solutions would be found.

When the placement of the partitions is finished simulated annealing is deployed. Simulated annealing in this implementation tries to improve the layout by switching locations of groups, and also by changing placement suggestions of either the n-part or p-part of the groups. The cost for each configuration is determined by the routing method described in 4.4, *Global Routing*. Hence, the cost of a placement is directly proportional to its routability and the quality of that routing solution. From the outset other cost functions were tried, since the global routing is a rather slow algorithm, but despite all effort no satisfactory function was found.

The version of simulated annealing implemented is not adaptive. This means that the user must provide values, based on the size of the problem, for all parameters used, such as for example the initial temperature. How this can be done is described in Chapter 6.

4.4 GLOBAL ROUTING

The aim of the global routing is to connect all nodes for each net to each other in such a way that no wires in one net crosses the wires of another net if the wires are in the same layer. Hence the problem of global routing can be described as finding the minimal spanning tree for each net in the circuit, while considering the solutions of all the other nets.

4.4.1 SOME APPROACHES ON GLOBAL ROUTING

To solve the global routing problem one of two different approaches can be used, the sequential or the concurrent. In the sequential approach the nets are routed one after the other. This can be very hard if there are many nets in the circuit (especially with the restriction imposed on the problem in this implementation, that a wire can not change between different layers). The order in which the nets are routed is crucial but even with best possible choice of order

it may be impossible to succeed without any crossings. Two ways to sequentially route multi terminal nets follow.

To find a routing solution to a multi terminal net, one approach is to extend algorithms intended for two terminal nets to route multi terminal nets. Such an approach is based on the maze routing algorithm. A maze router finds the shortest path between a start and an end node, utilizing an evenly spaced grid, where space available for routing is represented by unblocked vertices and obstacles are represented by blocked vertices. Starting in the start node a breadth-first search is done numbering the nodes in ascending order until the end node is reached. When this happens the algorithm backtracks a path until the start node is reached. A maze router extended to route multi terminal nets starts in all nodes in a net at the same time. The maze router makes it possible to choose pairs of nodes to connect, however, the solution will not be optimal. Another way is to begin with decomposing the multi terminal net into several two terminal nets and then let a maze or line probe router (a router similar to the maze router) connect the node pairs [6]. This approach also yields sub optimal results since the decomposition and the routing are totally separate from each other.

The concurrent approach takes all the nets into account at the same time. The only known technique to do this is to use integer programming. The resulting integer program is NP-hard [6].

In this thesis, a concurrent routing approach is done, hence an integer program is used.

4.4.2 USED APPROACH ON GLOBAL ROUTING

The routing will be carried out according to the model stated in chapter 3. An illustration of a net with its possible wires can be found in Figure 4.7. The number next to a node in the figure indicates the layer of that node.

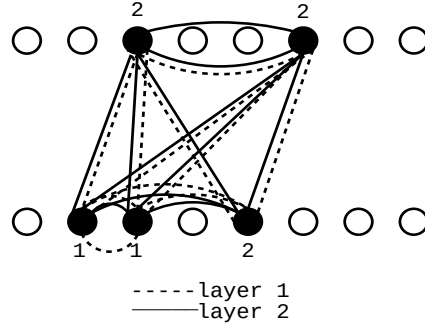


Figure 4.7: A net (the black nodes) with its possible wires

For the general routing model presented in chapter 3, the following 0/1 integer program can be formulated for concurrently routing all the nets.

x_{ijkn} - edge number k between vertices i and j in net n

c_{ijkn} - cost of edge x_{ijkn}

V_n - the set of vertices in net n

E_n - the set of edges in net n

N - the set of all nets

$$\min z = \sum_N \sum_{E_n} c_{ijkn} x_{ijkn}$$

The objective function (4.1)

$$\sum_{E_n} x_{ijkn} = |V_n| - 1 \quad n \in N$$

Completeness constraints (4.2)

$$\sum_S x_{ijkn} \leq |S| - 1 \quad \forall (S \subseteq V_n), n \in N$$

No subtours constraints (4.3)

$$\begin{aligned}
x_{ijkn} + x_{rstm} &\leq 1 && (i, j, k) \in E_n, n \in N \\
&&& \{(r, s, t) \in E_m \mid (r, s, t) \text{interferes with } (i, j, k)\} \\
&&& m \in N \setminus \{n\} \\
&&& \text{No crossings constraints}
\end{aligned} \tag{4.4}$$

$$\begin{aligned}
x_{ijkn} &\in \{0, 1\} && (i, j, k) \in E_n, n \in N \\
&&& \text{Binary constraints}
\end{aligned} \tag{4.5}$$

The first set of constraints means that of all possible edges of net n , exactly the number of vertices minus one edge, must be used. The second set of constraints means that of all subsets of vertices in net n with more than three vertices, the number of edges that connects those vertices must not be larger than the number of vertices in that subset minus one. These two sets of constraints combined mean that a spanning tree must be chosen for each net n .

The third set of constraints will generate, for the full adder, approximately 5000 constraints. One constraint for each pair of edges that, if both chosen, would interfere with each other. The constraints mean that at most one of two edges that would interfere with each other can be chosen at the same time.

This program can not be solved efficiently because of its size and NP-hardness, so instead the following heuristic that is loosely based on the lagrange relaxation of the program, is deployed.

- Define all possible wires for each net in both layers, also define an initial cost for each wire, e.g., the length of the wire and an extra factor proportional to the number of vias needed for that wire.
- Find the minimal spanning tree for each net.
- Find all crossings not allowed and add a penalty to the cost of each wire that is crossed. Wires not crossed receive a bonus instead.
- If there are any crossings go to the second step, else save the solution and restart with the achieved costs plus a small extra cost, based on length and vias, to find a better allowed solution.

In the second step an algorithm called Prim's algorithm, [3], is used and it can be described as follows. Start in an arbitrary vertex and add this vertex as the first vertex of a tree. Then select the cheapest edge that introduces a new vertex to the tree. Continue to select minimal cost edges that introduce new ver-

tices until all vertices are in the tree.

In order to verify the functionality and performance of the described heuristic approach, a lagrange relaxation, [5], of the problem has been formulated and solved. By relaxing the crossing constraints by “lifting” them into the objective function, and then minimizing the new objective function with only the completeness and subtour constraints in mind, an optimistic value of the original objective function can be attained. The lagrange relaxation method for the problem in this project can be described as follows.

First, formulate the new objective function, the lagrange function.

$$\min \sum_e c_e \cdot x_e - \sum_{ij} \mu_{ij} \cdot (1 - x_{ej} - x_{ei})$$

The lagrange function (4.6)

The parameters μ_{ij} are known as *lagrangians* or *lagrange multipliers* and it should be noted that they are not variables but parameters. By moving the crossing constraints to the objective function, every crossing results in an increase of the objective function with the μ for the violated constraint. The two other sets of constraints in the original formulation are still valid. The relaxed, far simpler problem, can now be solved by applying Prim’s algorithm to find a minimum spanning tree where the total cost of each edge is the sum of the edge’s cost, c_e , and the μ_{ij} belonging to each constraint involving that edge. This problem will be solved repeatedly for different values of μ . The start value for the μ_{ij} is often chosen to be zero, which means that the relaxed constraints are ignored. By choosing a value for UBD, upper boundary, and then do the following four steps repeatedly, an interval can be found in which the optimal value of the real objective function must lie. The UBD must be chosen so that there exists an allowed solution for the initial problem that has a value equal to or lower than the UBD.

- Solve the lagrange subproblem which results in a solution x^* and an LBD, a lower boundary.
- If x^* is an allowed solution to the original problem it is the optimal solution.
- Stop when the difference between UBD and LBD is sufficiently small or a given number of iterations is done.
- Update μ and proceed to the first step.

The update of the lagrangians is in this case done according to the polyak criterion.

$$\mu_{k+1} = \mu_k + t_k \cdot \gamma^k$$

Update of the lagrangians

(4.7)

$$\gamma^k = b - (A \cdot x)$$

The subgradient

(4.8)

$$t_k = \lambda_k \cdot \frac{UBD - h(\mu)}{\|\gamma^k\|^2} \quad 0 < \lambda_k < 2$$

The polyak criterion

(4.9)

As can be seen in (4.7), the new value of a lagrangian is chosen as the sum of the old value and the subgradient of the relevant constraint multiplied with the polyak criterion. In the case of this particular problem, the value of the subgradient adopts the value of -1 (none of the edges are part of the current solution), 0 (one of the edges is part of the current solution) and 1 (both of the edges are part of the current solution). The polyak criterion updates the value of the lagrangians so that their values level with the value of the lagrange function. The factor in the denominator is a scale factor so that the value of the lagrange function do not fluctuate too far from the UBD.

The goal of using the lagrange relaxation in this implementation is to evaluate how good the heuristic algorithm is. Hence, UBD is chosen as the value of the objective function produced by the heuristic, and only an LBD to compare UBD with is sought. The lagrange relaxed problem converge for smaller circuits, and, in these cases, the solution produced was the same for both the algorithms showing that the heuristic produced optimal results for the given objective function, for small circuits. In case of larger problems, e.g., a full adder circuit, the lagrange relaxed problem did not converge meaning that a duality gap exists. The solution produced by the lagrange method in this case was 30% lower than the solution from the heuristic method. Still, a good guess is that the heuristic produces a near optimal solution for this circuit, based on the observation that the solution produced by the lagrange method is virtually impossible to make allowed without ripping up and rerouting the entire net, even if the solution only violated one constraint. It should be mentioned that another reason that the lagrange relaxation algorithm was imple-

mented is that it could have been used, instead of the heuristic, to route the cells. However, since it turned out to be slower than the heuristic algorithm in its current form, and also unable to find usable solutions for larger cells, it was rejected as a router.

4.5 PIN PLACEMENT

Pin placement refer to the process of selecting the locations of the pins in the cell so that they can be reached from the outside by an automatic routing tool, simply by dropping a via. In order to achieve this a three step calculation is done right after global routing.

The first choice for a pin location will be a wire in the relevant net that is already placed in the routing metal layer by the router and runs in the channel.

Probably some of the nets which need pins will not have such a wire and therefore the next step will be to find an M-Poly contact on an appropriate node. Appropriate in this case is a node of the current net that has as much space as possible available. In this implementation this is seen as a node with as few wires as possible passing by, even though this is not always true.

If both of these two attempts fail a contact must be created. In that case the gate with the least number of wires passing by it will be equipped with an M-Poly contact. In this phase that means that the information that an M-Poly contact shall be connected to the gate will be bound to the object that represents that transistor.

5

DETAILED LEVEL DESIGN OF THE CELL

In this chapter the implementation of the program part where all the objects are placed in the layout is presented.

5.1 GENERAL

In the detailed model part of the program all the transistors, wires, etc. that until this moment only have had a schematic description, will be drawn and placed in the cell. This will be done so that all different objects, transistors, wires, etc., end up as close to each other as possible. During this phase, many geometrical rules have to be considered, especially as the program is intended to be usable also for future technologies, where entirely different rules may be crucial than for the technology, STMicroelectronics 0.18 μm , that was used as reference technology in the development of this program.

The creation of the layout will begin with the transistors, then the contacts, both the source/drain and M-Poly contacts, followed by vias that might be needed on supply nodes if the supply layer differ from the metal routing layer. If there are any M-Poly contacts that are not connected to a wire, i.e., they are placed only to provide pins, these contacts might need extra large metal parts not to violate a possible metal minimum area rule. After these metal parts are placed, all the wires are drawn. Finally, the supply wires, the cell layers, and substrate contacts are drawn.

Only placement of transistors and contacts and the routing will be described in detail.

5.2 PLACEMENT

Until now nothing has been said about the distances between the transistors. The calculation of these can be quite complicated, since they depend on a number of factors, e.g., whether two transistors next to each other have connected diffusions, if they in that case shall have a contact between them and also if there are any M-poly contacts connected to any of them, and in that case where. There are actually almost forty different cases of this kind. For each one of these cases there are also some subcases, e.g., whether the two transistors have the same width, whether the supply layer is in the first or second metal layer, which decides whether or not there is a need for extra vias on the supply nodes and whether any wire drawn from the source/drain contacts will expand through the channel. Furthermore, when calculating the distance for each of these subcases the one constraint that actually limits the distance can be one of many, depending on the technology used.

Before the placement of the transistors in the implementation described in this thesis the layout is empty. The transistors are placed one after the other first in the n-transistor row and then in the p-transistor row. They are at this moment just placed correctly in relation to the other transistors in the same row and will later, after some calculations based on for example the wire information and the appearance of the predrawn transistor rows, be redrawn in their correct places. The p-transistor row will be placed as far up and the n-transistor row as far down as possible and this will result in a maximal high channel between the two rows. They will also be horizontally centered in relation to each other so that the displacement of the source, drain and gate nodes compared to their relative positions in the high level model based part of the tool will be as small as possible. The distance to the left edge of the abutment box will be adjusted so that the two substrate contacts in the upper and lower left corner will get enough space. This is not done in an optimal way, which often will lead to an unnecessary wide cell.

When the transistors have been placed in their correct places, all the contacts will be placed. For each transistor first the possible source/drain contacts will be placed, then the possible gate contact (M-Poly contact connected to the gate) and, at last, the possible M-Poly contacts connected to the possible

source/drain contacts. For each M-Poly contact that is placed, a wire will be drawn, connecting the transistor to the contact. If two transistors of different width next to each other share a source/drain contact, the contact will be placed on the largest transistor.

Both transistors and contacts are placed layer for layer without using any pre-defined objects, in order to make the program as flexible as possible.

5.3 ROUTING

The detailed routing step follows the global routing, whose only goal was to decide which terminals should be connected and in which regions. In general problems, the majority of nets are of two terminal type. The netlist produced by the global router in this thesis always contains only two terminal nets, since multi terminal nets will be split into several two terminal ones. The different routing areas in this thesis is above the p-transistors, in the channel and below the n-transistors. These areas will be separately routed for the poly layer and the metal layer.

All but one of the detailed routing algorithms described in the literature studied for this project are based on a grid-based model and/or a reserved layer model. Since these two models will not be used in this implementation of reasons discussed in Chapter 3, only one algorithm remains and that is the river routing algorithm.

5.3.1 RIVER ROUTING

For reasons of simplicity it is common to work at a more abstract level than the actual layout when detailed routers are discussed. It is often sufficient to use a mathematical model for the wires and the rules they must obey. Wires are usually modelled as paths without any thickness, but the spacing between them is increased so that they fit in the layout anyway.

The river router works under the assumption that every net is of two-terminal type, all nodes lie on the boundary of the region, e.g., a channel, and that there are not any obstacles in the region that can block the wire. Such a routing problem can be seen in Figure 5.1 where the node pairs are marked 1, 2, ..., 5. Finally, it is assumed that no two wires belonging to different nets will need to cross each other if drawn correctly.

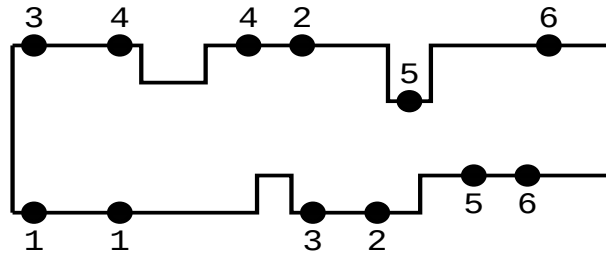


Figure 5.1: A channel with boundary

The river router follows the contour of the bounding box. At first the bounding box is just the channel and then it is updated after each wire is drawn so that the new boundary also includes the recently drawn wire.

In Figure 5.2 a half routed routing problem is shown. The current bounding box is marked with a thick line.

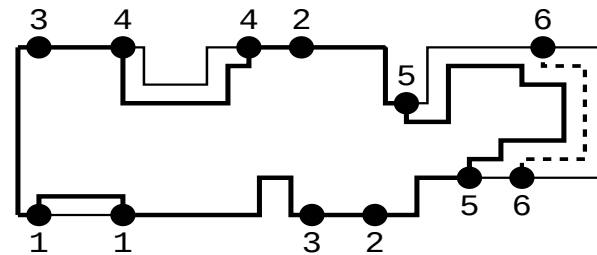


Figure 5.2: An updated boundary after half done river routing

The first thing that must be done is to decide in which order the wires shall be drawn. This can be done in several ways. A good approach might be to begin with connecting all the wires whose nodes are placed in the same row, since the channel crossing wires might cross these nodes otherwise. Hence in the example above the wires connecting the nodes labelled 4 and 1 will be drawn before the other ones. If there are several wires in this start group, these have to be mutually sorted. It is, of course, important to draw a wire whose nodes lie between two nodes of another wire before drawing this enclosing wire. When all wires connecting two nodes in the same row are drawn, the rest can

be drawn in the order in which the corresponding upper nodes are situated, starting either from the right or the left. Doing this, a routing as in Figure 5.3 is achieved.

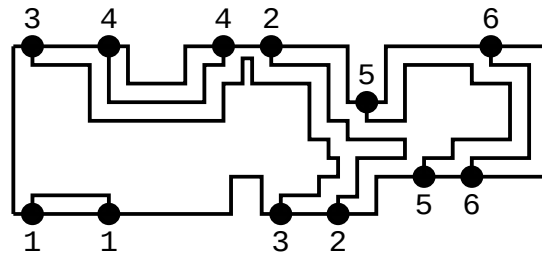


Figure 5.3: A routing problem routed by river routing

As can be seen in the figure, most of the drawn wires are unnecessary long. Therefore, the last step in the river routing algorithm aims at minimizing the corners by flipping them one after the other and checking that no design rules are broken. The final result of the routing will be very similar to the result shown in Figure 5.4.

5.3.2 DETAILED ROUTING IN THIS TOOL

The detailed routing of the channel in this implementation is principally based on the river routing algorithm, while the two other areas are routed by a simpler algorithm, covered later in this text.

In preparation for the river routing the following has been done in the detailed placement phase. For each of the objects drawn, in the transistors and contacts, an object has been added to one of four lists. These lists contain representations of the objects that might interfere with wires drawn in the channel in the poly layer or the metal layer, close to the n-row and the p-row. These representations are objects with some properties bound to them. The properties are the net name of the object so that wires belonging to the same net can pass through it, a lower boundary in case of the p-row lists and an upper boundary in case of the n-row lists. All objects also have a left and right boundary. The boundaries are calculated based on the layer and boundaries of the object that generates the interference. For example, a rectangle will in the active layer in the p-row have an interfering object in the p-row poly list, with the node property set to nil since no poly are allowed to cross it (since that

would create another transistor) and a lower boundary equal to its lower boundary minus a poly layer to active layer minimum separation distance. As a new wire is created, new objects that describe the new boundary created by it are added to the appropriate list. When the wires are drawn, a search is made for interfering objects in the relevant list and the wire is drawn in such a way that it follows these objects as close as possible.

The wires will, however, always move further out into the channel. This means that a wire will only follow the boundary as long as the boundary does not turn inwards again. This can clearly be seen in Figure 5.4 (take a look at the wire connecting the nodes labelled 3).

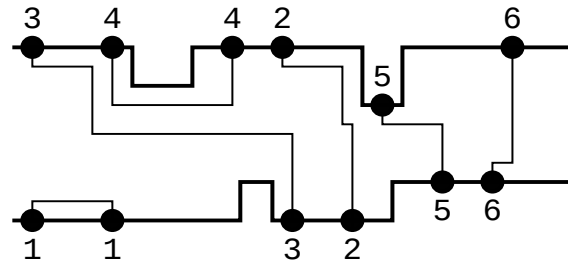


Figure 5.4: A finished routing

First, as has been suggested in section 5.3.1, all wires connecting nodes in the same row will be drawn. The order between these is decided by first choosing all wires that do not enclose any wires that are not already drawn. When this is done the chosen wires are drawn and the next set is found in the same manner until there are no more wires left to draw.

After this, the channel crossing wires are drawn. Wires whose upper node lies to the left of the lower will be drawn following the upper part of the boundary. The wires whose upper node lies to the right of the lower will be drawn following the lower part of the boundary. The wires will be sorted from right to left and then drawn. Wires whose two nodes lie right above each other can be drawn right away just crossing the channel.

The wire drawing function in SKILL has the peculiar property that in a wire drawn by it no single segment is allowed to be shorter than a certain distance. This makes it necessary to sometimes backtrack and correct a wire when an obstacle is encountered too close to another so that a segment of the resulting wire would be shorter than this distance.

The algorithm used for the wires running below the n-transistors and above

the p-transistors does not follow the contour of the boundary since this was deemed unnecessary, partly because the transistors are aligned to each other in these areas. Each wire is hence made up of two vertical and one horizontal segment. The horizontal segment will always be drawn as close to the transistor row or, if existing, the closest underlying wire, as possible. The drawing order is made up in the same way as for the wires in the channel whose nodes lie in the same row.

The last routing done is to connect the supply wires to the appropriate nodes on the transistors.

The right edge of the supply wires will be adjusted to the cell library grid, which is a special grid that will be used by the external router that will connect the standard cells into larger layouts. This may lead to an unnecessary large gap between the right edge of the rightmost transistor and the abutment box, i.e., the cell might be wider than necessary.

6

RESULTS

6.1 THE RESULTING PROGRAM

This thesis has resulted in a program that takes the following input arguments:

- The name intended for the *cellview* where the resulting cell shall be drawn.
- The name of the *library* where the cellview shall be put.
- The *schematic list*.
- The *input nodes*, i.e., the nets to which a terminal of I/O-type ‘input’ shall be connected.
- The *output nodes*, i.e., the nets to which a terminal of I/O-type ‘output’ shall be connected.
- The *input/output nodes*, i.e., the nets to which a terminal of I/O-type ‘inputOutput’ shall be connected.
- A *flag* that decides whether to keep the order and the source-drain rotations of the transistors in the schematic list.

The schematic list will contain one list for each transistor in the schematic. Each of these lists will contain information on:

- The *name* of the transistor (optional). Default: nil.

- The *type* of the transistor [*P* or *N*].
- The node name of the *drain* of the transistor.
- The node name of the *gate* of the transistor.
- The node name of the *source* of the transistor.
- The *width* of the transistor (optional). Default: minimum width from the techfile, the file containing all technology specific parameters provided by the manufacturer of the IC, multiplied with the driving parameter.
- The *length* of the transistor (optional). Default: minimum length from the techfile.
- The *rotation* of the transistor [*sd* (source-drain) or *ds* (drain-source)] (optional). Default: *ds*.

The flag that decides whether or not to use the configuration of the transistors in the schematic list, i.e., skip the placement step, is mainly intended as a way of reusing previous solutions. This feature is usable for two reasons. Firstly, the placement phase of the program is quite time consuming compared to the rest of the program and hence a lot of time will be saved by being able to reuse a placement solution. Secondly, since the simulated annealing algorithm used in the placement phase is non deterministic, the same input arguments will often yield different results, especially for larger cells where global optimum can not be reached. Hence, if a good solution is generated for a cell at one time it is not sure that the next time the program is run an equally good solution, regarding for example channel congestion, will be found. If for instance three full adders with three different driving capabilities are sought, it is probably best to use one good schematic and just reroute the cell two times with different driving settings. Yet another reason might be that a user would like to make her own placement.

There are two files of certain interest in the program, the *Rules file* and the *Main file*.

In the *Rules file*, the parameters needed to follow the rules discussed in Chapter 2.4 are set, as well as the driving of the cell, the layer of the supply wires, and algorithm specific parameters. A typical *Rules file*, setup for 0,35 μm can be found in APPENDIX A.

Some of the parameters in the *Rules file* need special attention. These are those which regulate the simulated annealing algorithm. The simulated

annealing parameters are, *StartTemp* (initial temperature), *EndTemp* (end temperature), *NumberOfMovesPerIter* (number of moves per iteration), and a *TurningTemp* (speed up parameter) that decides when to speed up the temperature drop process. *StartTemp* should be chosen so that the deterioration ratio in the beginning of the algorithm is 80%. By setting the parameter *PrintTempAndRateInfo* to true, the ratio information will be displayed in each iteration. *EndTemp* should be chosen so that no cost increasing moves are accepted in the final iterations of the algorithm. *NumberOfMovesPerIter* must be chosen so that thermal equilibrium is reached for each temperature step. All these parameters increase with the size of the problem. A good choice for *TurningTemp* might be to let the process run faster the last 33% of the total number of temperature steps. As long as the cells generated are of similar size, the parameters do not need to be changed. Also if *StartTemp* and *NumberOfMovesPerIter* are chosen sufficiently high, and if *EndTemp* is chosen sufficiently low, they will never need to be changed at all. This will however lead to increased running times.

More deliberate ways to choose the simulated annealing parameters can be found in [7]. The remaining parameters in the *Rules file* are covered in APPENDIX A.

The *Main file* will, when loaded, load all the other files, eight altogether including the rules file and it will define a main procedure called *CreateCell()*. By calling *CreateCell()* with the correct input arguments, as discussed above, a cell will be created according to the values in the rules file. This means, that the *Rules file* must be properly set up before loading the main file. Alternatively the *Rules file* can be loaded separately after each change.

The program will always return a list of the resulting placement of the transistors so that the solution can be reused.

The layout generated by the program always has to be checked to ensure a working cell. If the global router did not succeed in finding a solution without any crossings the program will warn the user, but if the layout generation fails due to for example an insufficient cell height no warning will be sent. Also, before the layout can be used, it has to be checked for notches by the user.

For more detailed instructions on how to use the program, refer to APPENDIX B.

6.2 TEST RESULTS

A full adder, a dynamic latch, a transmission gate, a CMOS OR-gate, and an XOR2-gate have been generated by the program and all have been successfully placed and routed. The program has been proven to work in both STMicroelectronics 0.18 μm and AMS 0.35 μm technologies. A generated XOR2-gate in 0.35 μm can be seen in Figure 6.1.

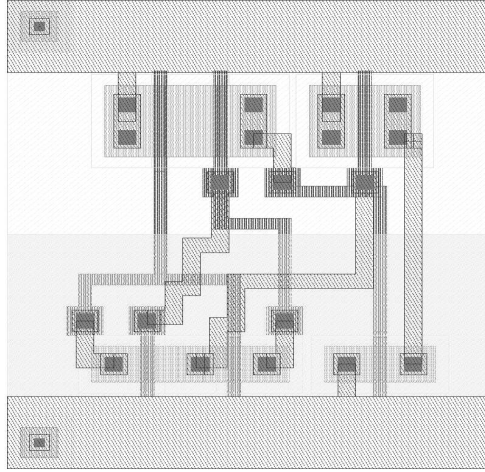


Figure 6.1: Layout of an XOR2-gate in AMS 0.35 μm

When fed the schematic in Figure 4.1, represented by the following list (the schematic list described in section 6.1, where no optional parameters are specified):

```
'(( N S l gnd)(N l C j)(N j A k)(N k B gnd)(N f B gnd)(N f A gnd)(N f C gnd)(N l X f)(N
X C c)(N X A d)(N d B gnd)(N c B gnd)(N c A gnd)(N C 0 X gnd)(P S l vdd)(P g C
vdd)(P g A vdd)(P g B vdd)(P h A g)(P i B h)(P l C i)(P l X g)(P X C a)(P a B vdd)(P
a A vdd)(P b B a)(P X A b)(P C 0 X vdd))
```

the program generates the layout in Figure 6.2 and Figure 6.3 respectively, depending on which technology the *rules file* was setup for. The transistor ordering was generated once by the program and then the routing was done for both technologies. As can clearly be seen in the figures the global routing is the same, only the detailed routing differs. Noteworthy is also the relative width of the routing layers. In 0.18 μm one of the layers are clearly much wider than the other, the thin layer is the poly-layer and the other the metal-layer. In 0.35 μm both the routing layers are almost of the same width. This is

due to the fact that the poly layer is scaled down more than the metal-layers as the technologies shrink. It can also be clearly seen that the contacts are relatively much larger in 0.35 μm than in 0.18 μm . This means that a larger part of the cells in 0.35 μm is used for contacts which makes it more pressing to keep the number of contacts low.

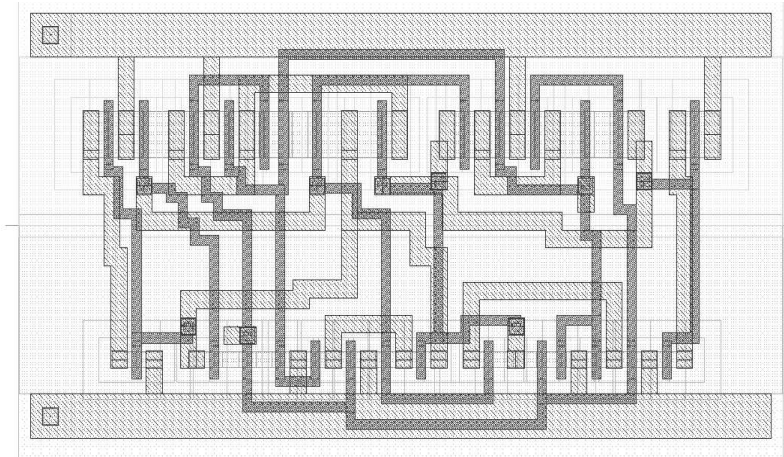


Figure 6.2: Layout of a full adder generated by the program in STMicroelectronics 0.18 μm

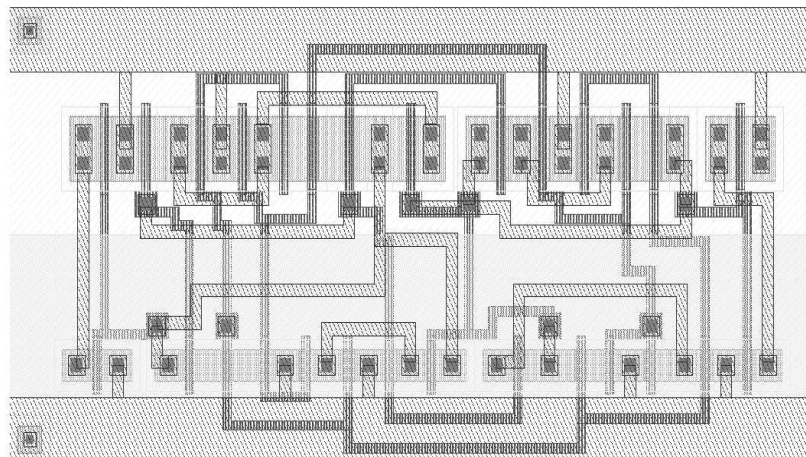


Figure 6.3: Layout of a full adder generated by the program in AMS 0.35 μm

Both layouts have passed both a DRC (Design Rule Check) and an LVS-check (Layout Versus Schematic check). Only three notches were noted in

both the full adder layouts while in the case with the XOR2-gate none was found. This is generally the case with the smaller cells generated. As long as the cells are smaller than a latch, notches are unlikely to occur.

Two small cells from a hand crafted cell library in AMS 0.35 have been compared with the same cells generated by this tool. The tool successfully generated a layout in the same height. However, the cells were, of reasons discussed in chapter 5.2 and 5.3, wider than necessary. This is, at least when it comes to small cells, easily manually rectified. The exact differences in the layouts could not be examined, since the hand crafted cells only were represented by an abstract. The compared cells were fetched from the AMS 0,35 μm HRDLIB library.

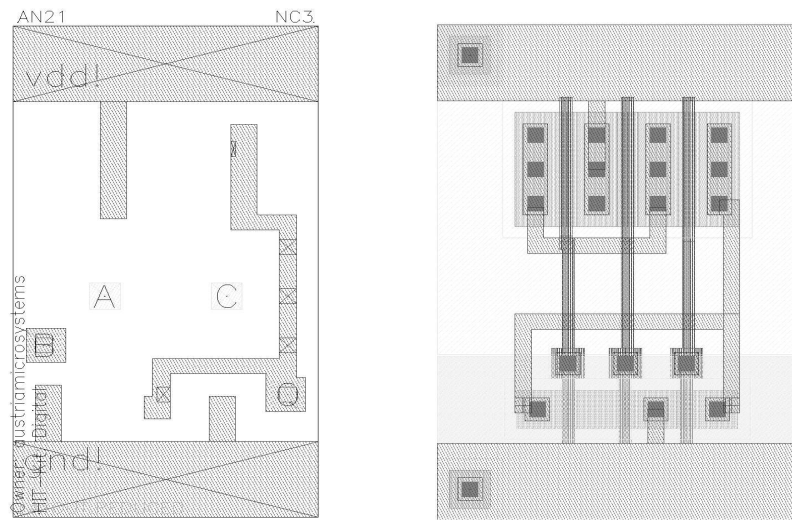


Figure 6.4: A2NO2 to the left, the generated version to the right

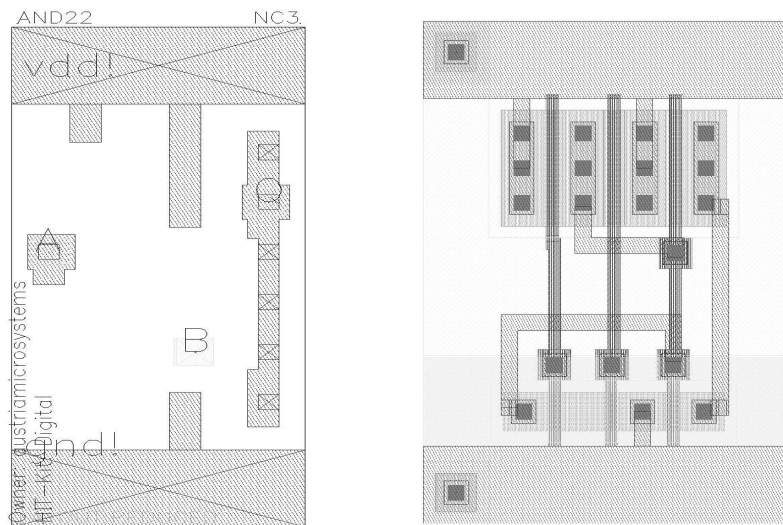


Figure 6.5: AND22 to the left, the generated version to the right

Notice the unnecessary contact in the generated cell to the left in Figure 6.5. This is because the parameters in the global router are not optimally set up, since this is very hard to do.

Finally, it should be mentioned that a number of tests have been conducted where the length and width of individual transistors have been changed. All layouts generated during these tests came out well. However, it is of course hard to test every possible case to verify that there are no bugs in the program, neither has this been done. To illustrate the ability of the program to generate cells with different driving capabilities the nonsense example in Figure 6.6 has been generated.

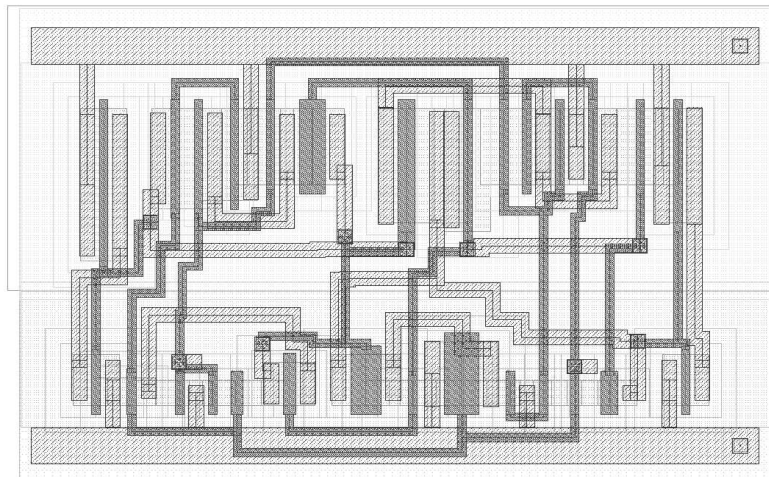


Figure 6.6: The full adder with modified transistor lengths and widths

7

CONCLUSIONS

7.1 SUMMARY

To begin with the cell is modelled as two rows of transistors, where each of the transistors is modelled as three, on each other following, evenly spaced nodes, as shown in Figure 3.1. Running between these nodes are all possible wires that can be chosen to connect each net. These wires may cross each other only when they lie in different layers or when they belong to the same net. The wires run above the p-transistor row, below the n-transistor row, and in the channel formed by the two rows, as shown in Figure 4.7.

The tool generates the cells in a number of steps. These are partitioning, placement, routing, pin placement, and physical layout.

Many different approaches for partitioning exist. These are often well-suited for general problems, but since the problem solved in this thesis has some usable properties, such as a CMOS structure, a fast, good and still easier algorithm is used.

Also for the placement problem there are several approaches. In this project, simulated annealing is used along with the routing algorithm for cost evaluation.

The routing of multi-terminal nets can be done in some different ways. However, if many multi-terminal nets are to be routed concurrently, only an integer program formulation of the problem works. This problem is NP-hard and, therefore, some sort of relaxation or heuristic method must be used. The method used in this project is based on and verified by lagrange relaxation.

In the layout part, the physical aspects are added to the model. This includes separation rules, minimum widths, etc. The transistor placement is fixed so that transistor gates can only be placed vertically. Notches are not taken into consideration.

In the layout phase, the design rules are taken into consideration. First, all the transistors are placed in two horizontally centred, parallel rows. Then all the contacts and vias are placed. The result is routed according to the solution produced by the global router, using a river router in the channel, and a simpler algorithm above and below the channel.

7.2 DISCUSSION OF THE RESULT AND POSSIBLE IMPROVEMENTS

The objectives stated in the beginning of this thesis have been met. The project has resulted in a working program that creates standard cells according to a given schematic. The program is parameterizable and has been proven to be able to shift effortlessly between different technologies, driving capabilities, cell heights etc.

There are some things that can be improved in the program. The program is currently implemented as a number of steps which are executed in a strict sequential manner. In every step a limited number of factors are taken into account. For example the widths and lengths of the transistors are disregarded in all the global algorithms. This in order to make each step less complex and also because some parameters, that maybe ought to have been considered, are dependent on results produced in later steps. In the global routing step for instance, the real positions of the nodes are unknown, which might lead to that a wire is estimated to be longer than another wire even though this might not be the case when the layout is generated. A more optimal result might be found if the program iterated between different steps, and also if more factors were considered in each step.

All cells generated have been successfully routed, generally with good results. There are some improvements that could be done to the routing heuristic. Some of the now used cost parameters could for example probably be tweaked so that the heuristic gives a more optimal result. Furthermore, the channel congestion is disregarded during the entire implementation. The natural way to take it into account would be to add congestion constraints to the NP-problem formulated during global routing. If they were included, the

used heuristic would have to be modified to penalize not only the wires that violate the crossing constraints, but also those who violate the congestion constraints. This modification would be rather easy to implement, but we do not know whether the heuristic still would be convergent. Noteworthy is also that the channel congestion constraints can never be totally accurate with less than that the exact positions of all transistors are known. Therefore, a good modification would be to generate the layout, or at least calculate the real positions of all transistors, before doing the routing.

As mentioned earlier, transistors are currently neither rotated nor split. This means that if the driving of a cell or the width of an individual transistor is specified too large relative the cell height, the program will fail. If the above proposed modification to the routing heuristic is implemented, then both splitting and rotation of transistors could be handled in the placement step. This because the heuristic is used to estimate the quality of the placement, and would yield a high cost for placements that violate the congestion constraints, resulting in that these placements would be sorted out. If no placement that fulfils the congestion constraints is found after a certain number of iterations splitting and rotation of transistors could be tried. Another solution might be that the user splits large transistors already in the schematic.

It should be mentioned that the modifications proposed above would result in greatly increased running times for the program, but as we have been told this is of no great concern.

Many algorithms used during the development of this tool have not been fetched from books covering electronics. They have either been developed specifically for this tool or found in books covering more mathematical subjects such as optimization and computer science.

We are satisfied with the result of this project and, as previously stated, the tool generates fully functional standard cells. However, it is our strong belief that the tool should be further developed, before it is used as a standard cell generator. If this is done, we are convinced that the tool could be very powerful.

REFERENCES

- [1] Bäckström, Karl-Johan: *Diskret matematik*. Studentlitteratur 1986. ISBN: 91-44-22641-1
- [2] Hjalmarson, Emil: *Introduktion till skill*. 2002.
- [3] Foulds, L.R: *Optimization Techniques*. Springer-Verlag 1981. ISBN: 0-387-90586-3
- [4] Kang, Sung-Mo / Leblebici, Yusuf: *CMOS Digital Integrated Circuits*. McGraw-Hill 1999. ISBN: 0-07-116427-8
- [5] Lundgren, J. / Rönnqvist M. / Värbrand M.: *Optimeringslära*. Studentlitteratur 2003. ISBN: 91-44-04205-1
- [6] Sherwani, Naveed A.: *Algorithms for VLSI Physical Design Automation 3rd ed*. Kluwer Academic Publishers 1999. ISBN: 0-7923-8393-1
- [7] van Laarhoven, P.J.M. / Aarts E.H.L: *Simulated Annealing: Theory and Applications*. D. Reidel Publishing Company 1987. ISBN: 90-277-2513-6
- [8] *Back-end Standards for Unicaad HCMOS8D Libraries*. UNICAD 2.4 2000.

APPENDIX A

THE RULES FILE

The file referred to as the *Rules File* contains all rules discussed in Chapter 2.4. Furthermore it contains parameters needed for the simulated annealing procedure, name of layers, location and name of the technology file, etc.

Here follows the *Rules File* setup for AMS 0.35 μm technology.

```

;-----
;  TECHFILE PARAMETERS
;-----

TechFilePath    = "TECH_CSI"    ;Tech. file location
TechFileName    = "techfile.cds" ;Tech. file name

;Tech. file names of layers etc.
PolyLayer       = "POLY1"       ;'poly' layer for routing and gates
MetalLayer      = "MET1"        ;'metal' layer for routing
Metal2Layer     = "MET2"        ;Secondary metal layer
PdiffLayer      = "DIFF"        ;Pdiff layer
NdiffLayer      = "DIFF"        ;Ndiff layer
NwellLayer      = "NTUB"        ;Nwell layer
ContactLayer    = "CONT"        ;Contact layer poly->metal
ViaLayer        = "VIA"         ;Contact layer metal->metal2
NtapName        = "ND_C"        ;Substrate contact used in the p-type area
PtapName        = "PD_C"        ;Substrate contact used in the n-type area
PinLabelLayer   = list("text" "drawing") ;Pin label layer
PinLabelHeight  = 1.0           ;The height of the pin label font

;-----
;  CELL PARAMETERS
;-----

Driving         = 1             ;The driving ratio to a min. sized transistor cell

```

GridSpacing = 1.0 ;[um] Grid spacing (the cell width will be a multiple
 ;of this width). If set to Mfg the generated cell will
 ;be as narrow as possible.
 CellHeight = GridSpacing*13 ;[um] Height of the cell
 SupplyWidth = 2.0 ;[um] Width of the supply wires
 SupplyLayer = MetalLayer ;Layer for power supply
 P_N_Ratio = 2 ;The width ratio between P- and N-type transistors
 ;with the same driving capability
 NBound = CellHeight/2.0 ;[um] The height of the n-type transistor area, this
 ;value is used when the cell layers are drawn
 MinStartOrigin = GridSpacing ;[um] The smallest allowed distance from the Y-
 ;axis to the diffusion of the first transistor
 NdiffXStartDueToPTap = 1.9 ;[um] PTap width + PTapNdiffSep, sees to that the
 ;n-transistors don't end up to close to the
 ;substrate contact
 PdiffXStartDueToNTap = 1.9 ;[um] NTap width + NTapPdiffSep, sees to that the
 ;p-transistors don't end up to close to the
 ;substrate contact
 PowerName = "vdd!" ;The name used for the power supply pin
 GroundName = "gnd!" ;The name used for the ground supply pin

;List over the layers that shall cover the n- and p-areas respectively.

;Defined as: list("layer" rightenc leftenc upperenc lowerenc type)
 CellLayers = list(list("NTUB" 0.0 .0 1.0 nil 'P)
 list("NPLUS" 0.0 0.0 .0 -SupplyWidth 'N)
 list("PPLUS" 0.0 0.0 -SupplyWidth .0 'P)
) ;end list

;-----
 ; LAYOUT PARAMETERS THAT WERE NOT FOUND IN THE TECHFILE
 ;-----

;Special rules (These rules might be fetchable in the tech. file, but we haven't found them)

PolyCornerPdiffSep = .3 ;[um] Isn't used if this field is nil
 PolyCornerNdiffSep = .3 ;[um] Isn't used if this field is nil
 MetalMinArea = 0 ;[um]

```

DiffMinArea      = 0           ;[um]
MetalViaEnc      = .2         ;[um]
Metal2ViaEnc     = .1         ;[um]
NdiffContactEnc  = .3         ;[um]
PdiffContactEnc  = .3         ;[um]
PolyContactEnc   = .2         ;[um]
MetalContactEnc  = .1         ;[um]
ABoxPolyEnc      = .3         ;[um] ABox - Abutment box
ABoxMetalEnc     = .3         ;[um]
AboxNdiffEnc     = .3         ;[um]
AboxPdiffEnc     = .3         ;[um]
MinGateSep       = .7         ;[um]
SDExt            = 1.100000    ;[um] The source- and drain-extension from the
                                ;side of the gate
GateExt          = 0.400000    ;[um] The gate-extension from the upper/lower
                                ;border of the source/drain

```

```

;Transistor information

```

```

TransistorMinWidth = 1         ;[um] Is set to NdiffMinWidth if this field is nil
TransistorMinLength = .3       ;[um] Is set to PolyMinWidth if this field is nil

```

```

N_Layers = '( ("NPLUS" 0.30000) ) ;The layers that should cover the n-type transis-
                                ;tors with the defined overlap

```

```

P_Layers = '( ("PPLUS" 0.30000) ) ;The layers that should cover the p-type transis-
                                ;tors with the defined overlap

```

```

;-----
;  INFORMATION FROM THE TECHFILE
;-----

```

```

;Load the tech. file

```

```

tfid = if( TechFilePath == "" || TechFileName == "" then
    println("No tech. file and/or path to tech. file defined.")
    println("Loading default tech. file.")
    techOpenDefaultTechFile()
else
    techOpenTechFile(TechFilePath TechFileName "r")

```

```
) ;end if
```

```
;Manufacturing grid resolution
```

```
MfgGridRes      = techGetMfgGridResolution(tfid)
```

```
;Spacing rules
```

```
PolyWidth        = techGetSpacingRule(tfid "minWidth" PolyLayer)
```

```
MetalWidth       = techGetSpacingRule(tfid "minWidth" MetalLayer)
```

```
Metal2Width      = techGetSpacingRule(tfid "minWidth" Metal2Layer)
```

```
ContactWidth     = techGetSpacingRule(tfid "minWidth" ContactLayer)
```

```
ViaWidth         = techGetSpacingRule(tfid "minWidth" ViaLayer)
```

```
MetalMetalSep    = techGetSpacingRule(tfid "minSpacing" MetalLayer)
```

```
PolyPolySep      = techGetSpacingRule(tfid "minSpacing" PolyLayer)
```

```
;PolyContactSep = techGetSpacingRule(tfid "minSpacing" PolyLayer ContactLayer)
```

```
PolyContactSep = 0.4
```

```
PdiffPdiffSep    = techGetSpacingRule(tfid "minSpacing" PdiffLayer)
```

```
NdiffNdiffSep    = techGetSpacingRule(tfid "minSpacing" NdiffLayer)
```

```
NdiffContactSep = 0.5 ;techGetSpacingRule(tfid "minSpacing" NdiffLayer Contact-  
Layer)
```

```
PdiffContactSep = 0.5 ;techGetSpacingRule(tfid "minSpacing" PdiffLayer Contact-  
Layer)
```

```
ContactContactSep = techGetSpacingRule(tfid "minSpacing" ContactLayer)
```

```
NwellPdiffEnclose = techGetOrderedSpacingRule(tfid "minEnclosure" NwellLayer  
PdiffLayer)
```

```
PolyNdiffSep =
```

```
    if( PolyCornerNdiffSep then
```

```
        PolyCornerNdiffSep
```

```
    else
```

```
        techGetSpacingRule(tfid "minSpacing" PolyLayer NdiffLayer)
```

```
    ) ;end if
```

```
PolyPdiffSep =
```

```
    if( PolyCornerPdiffSep then
```

```
        PolyCornerPdiffSep
```

```
    else
```

```
        techGetSpacingRule(tfid "minSpacing" PolyLayer PdiffLayer)
```

```
    ) ;end if
```



```

;Transistor min. size
unless( TransistorMinWidth
          TransistorMinWidth = techGetSpacingRule(tfid "minWidth" NdiffLayer)
) ;end unless
unless( TransistorMinLength
          TransistorMinLength = PolyWidth
) ;end unless

```

```

;-----
;  ALGORITHM PARAMETERS
;-----

```

```

;Simulated annealing
StartTemp      = 5           ;Initial temperature (this value should be cho-
                             ;sen so that the initial acceptance ratio is about
                             ;80%)
EndTemp        = .01        ;End temperature (chosen so low that accept-
                             ;ance ratio have been 0 for a couple of iterations
                             ;before this temperature)
TurningTemp    = 1           ;After this point the cooling process is sped up
NumberOfMovesPerIter = 25    ;Number of moves to thermal equilibrium for each
                             ;temperature
GroupSwapProb  = 31          ;[%] The probability that we will exchange places
                             ;between 2 groups rather than exchange one half
                             ;of a group. If set to 0 no group exchanges will
                             ;occur.

PathSearchIterLim = 25      ;The maximum number of iterations allowed with-
                             ;out an improvement in the global routing
                             ;algorithm

PrintTempAndRateInfo = t    ;[true/false] Whether or not to print 'acceptance
                             ;ratio' and 'temperature' in simulated annealing

```

```
;-----  
;  ASSORTED PARAMETERS  
;-----  
  
LargeNumber    = 10000000000000;Internal parameter DO NOT CHANGE!  
DeltaError      = 1/1000000.0    ; Internal parameter DO NOT CHANGE!  
  
;The SKILL procedure rodCreatePath() complains (says the path would be selfinter-  
;secting) if it gets two points that lies nearer each other than the distances below. We  
;usually set these to the minimum widths of the layers involved, the true values are not  
;known.  
SelfintersectDistPoly  = PolyWidth ;[um]  
SelfintersectDistMetal = MetalWidth;[um]
```

APPENDIX B

GETTING STARTED

This appendix shortly covers how to get started with the program *CreateCell()*.

- 1) Start by opening the file named *Rules.il*. Here you find all the parameters that are used in the cell construction. Change them according to your technology.
- 2) Now you have to define a schematic. A schematic is a list of lists that represent the transistors in the cell you want to create. A list representing a transistor looks like (the parameters inside brackets are optional):

```
'( [Name {symbol}]
  Type {P or N, symbol}
  DrainNode {symbol}
  GateNode {symbol}
  SourceNode {symbol}
  [ChannelWidthFactorVsDefaultWidth {float}]
  [ChannelLengthFactorVsDefaultLength {float}]
  [Rotation {ds or sd, symbol}]
)
```

e.g., *Inv = '((P Out In vdd) (N Out In gnd))* represents an inverter. Power has to be denoted *vdd* and ground *gnd*.

- 3) Load the file named *Main.il* in Cadence.
- 4) The program is executed by the command:

```
CreateCell( Schematic
            InputNodes
            OutputNodes
            InputOutputNodes
            CellLibrary
            InstanceName
            KeepFlag
          ) ;end CreateCell()
e.g., CreateCell( Inv '(In) '(Out) nil "Test" "Inv" nil )
```

NOTE: The `KeepFlag` argument decides whether to change the order and rotation of the transistors in Schematic or to keep it as it is. This can be a usable feature if you want to reuse a previous solution.

Besides creating a layout of the cell, *CreateCell()* will return a list, named *ReturnList*, containing the sorted schematic, so that the solution can be reused.

På svenska

Detta dokument hålls tillgängligt på Internet – eller dess framtida ersättare – under en längre tid från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns det lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>

In English

The publishers will keep this document online on the Internet - or its possible replacement - for a considerable time from the date of publication barring exceptional circumstances.

The online availability of the document implies a permanent permission for anyone to read, to download, to print out single copies for your own use and to use it unchanged for any non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional on the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its WWW home page: <http://www.ep.liu.se/>

© Terese Ekebrand and Nils Funke